



JAVA BACKEND

TECHNICAL LEAD INTERVIEW GUIDE

140 questions, answer checkpoints, system design drills, and a focused preparation plan

For engineers with 10+ years of experience in Java, Spring Boot, microservices, AWS, relational databases, Kafka, and CI/CD

PREPARATION EDITION



How to use this guide

At Technical Lead level, interviewers are rarely testing recall alone. They are looking for judgment: how you clarify an unclear problem, select trade-offs, protect production, communicate risk, and help a team deliver. Use each question as a prompt for a structured, evidence-based conversation.

A strong 3-part answer

- Start with context and constraints. State scale, reliability, consistency, security, cost, and team assumptions before choosing a solution.
- Explain the decision and trade-offs. Compare realistic options, choose one, and say what would cause you to choose differently.
- Close with evidence and operations. Describe testing, rollout, metrics, failure handling, and a real result when possible.

Depth signals expected from a Technical Lead

Signal	What good evidence sounds like
Ownership	I made the decision, aligned the owners, defined rollback, and stayed with the result.
Trade-offs	Option A optimized latency but increased operational load; we chose B because the SLO did not justify that cost.
Scale	I estimated traffic, concurrency, storage growth, failure domains, and the bottleneck before naming technology.
Safety	We used compatibility, canarying, metrics, and a tested recovery path instead of relying on confidence.
Leadership	I created clarity and enabled other engineers; I did not personally become the critical path.

STAR+ for leadership stories

Situation and stakes -> Task and your ownership -> Actions and decisions -> Result with measures -> Reflection: what you learned and what changed afterward.

Contents

How to use this guide	2
Contents	3
Technical Leadership and Architecture	4
Java Language and Core Engineering	5
JVM, Concurrency, and Performance	6
Spring and Spring Boot	7
Microservices and Distributed Systems	8
API Design and Security	9
System Design	10
AWS Cloud Architecture	11
Relational Databases and Persistence	12
Kafka and Event-Driven Systems	13
CI/CD, Containers, and Platform Engineering	14
Reliability, Observability, and Testing	15
Coding and Problem Solving	16
Behavioral, People, and Delivery Leadership	17
End-to-End System Design Drills	18
Self-Scoring Rubrics	20
Four-Week Preparation Plan	21
Final Interview Readiness Checklist	22

Technical Leadership and Architecture

1. How do you turn an ambiguous business goal into an executable technical plan?

Strong answer should cover: Clarify outcomes and constraints; identify risks and assumptions; propose options; sequence thin vertical slices; define measurable success and decision owners.

2. How do you decide whether to build, buy, or reuse an existing platform capability?

Strong answer should cover: Compare strategic differentiation, total cost, time-to-value, integration risk, lock-in, security, operations, exit cost, and opportunity cost.

3. Describe an architecture decision you reversed. What evidence changed your mind?

Strong answer should cover: Use a real decision; explain original context, new evidence, reversible steps, stakeholder communication, migration safety, and learning without blame.

4. How do you run an architecture review without becoming a delivery bottleneck?

Strong answer should cover: Use lightweight decision records, risk tiers, time-boxed reviews, paved roads, explicit guardrails, and delegated ownership.

5. How do you balance feature delivery with technical debt?

Strong answer should cover: Quantify user and operational impact; distinguish debt from preference; reserve capacity by risk; attach debt reduction to product outcomes; track trends.

6. What should a useful architecture decision record contain?

Strong answer should cover: Context, forces, considered options, decision, consequences, owner, date, status, evidence, rollback or revisit triggers.

7. How do you establish engineering standards across several teams?

Strong answer should cover: Co-create a small baseline; automate checks; provide examples and migration paths; allow justified exceptions; measure outcomes, not conformity.

8. How would you handle a senior engineer who strongly disagrees with your design?

Strong answer should cover: Seek the underlying concern; compare evidence against agreed criteria; run a spike if needed; document the decision; disagree and commit.

9. How do you estimate and communicate technical risk to non-technical leaders?

Strong answer should cover: Translate into probability, impact, exposure window, leading indicators, options, cost of delay, and a clear recommendation.

10. What metrics tell you that your technical leadership is effective?

Strong answer should cover: Delivery predictability, change failure rate, recovery time, escaped defects, reliability, developer flow, ownership health, and business outcomes.

Java Language and Core Engineering

11. Explain the equals and hashCode contract and a production failure caused by violating it.

Strong answer should cover: Cover reflexive/symmetric/transitive/consistent behavior, null, equal hash codes, mutable keys, records, and HashMap/HashSet symptoms.

12. When would you choose a record, immutable class, sealed hierarchy, or ordinary mutable class?

Strong answer should cover: Discuss domain semantics, invariants, identity, framework constraints, serialization compatibility, pattern matching, and controlled mutation.

13. What are the practical differences among List, Set, Map, and concurrent collection implementations?

Strong answer should cover: Compare ordering, uniqueness, complexity, memory, iteration consistency, atomic operations, contention, and real access patterns.

14. How do generics provide type safety, and what problems remain because of type erasure?

Strong answer should cover: Cover invariance, PECS, bounds, wildcards, bridge methods, reifiable types, heap pollution, and runtime limitations.

15. How would you design an immutable Java type that contains collections?

Strong answer should cover: Final state, constructor validation, defensive copies, unmodifiable views versus copies, safe publication, and no mutable leakage.

16. When do streams improve code, and when do they make it worse?

Strong answer should cover: Compare declarative transformations with stateful logic; side effects; short-circuiting; allocation; debugging; parallel-stream hazards.

17. How do Optional, null, and exceptions differ as API design choices?

Strong answer should cover: Use Optional mainly for optional return values; distinguish absence from failure; avoid Optional fields/parameters unless justified; preserve clear contracts.

18. Explain checked versus unchecked exceptions and your policy for a service codebase.

Strong answer should cover: Discuss recoverability, boundary translation, stable error models, causal chains, retryability, logging ownership, and avoiding catch-all handling.

19. What Java serialization risks have you encountered, and what do you use instead?

Strong answer should cover: Mention security, fragile binary compatibility, gadget chains, versioning, explicit schemas, safe serializers, and allowlists when legacy use is unavoidable.

20. How do you review Java code for correctness and maintainability?

Strong answer should cover: Start with contracts and failure modes; inspect state and concurrency; simplify control flow; verify naming, tests, security, observability, and performance evidence.

JVM, Concurrency, and Performance

21. Explain stack, heap, metaspace, direct memory, and thread-local storage in a JVM process.

Strong answer should cover: Tie each area to lifecycle, allocation, limits, failure modes, container memory, native overhead, and the measurements used to diagnose it.

22. How does garbage collection affect latency and throughput?

Strong answer should cover: Explain reachability, allocation rate, pauses, concurrent work, promotion, fragmentation, collector goals, heap sizing, and evidence from GC logs.

23. A service has rising memory use. How do you distinguish a leak from healthy caching?

Strong answer should cover: Observe after-GC trend; compare live set and allocation rate; inspect heap histogram/dump; find retaining paths; validate cache bounds and eviction.

24. Explain happens-before and safe publication under the Java Memory Model.

Strong answer should cover: Cover visibility and ordering; synchronized/locks, volatile, final-field guarantees, thread start/join, concurrent collections, and unsafe publication examples.

25. When is volatile sufficient, and when do you need locking or atomic operations?

Strong answer should cover: Volatile supports visibility and simple state; compound invariants require atomicity; discuss CAS contention, locks, and higher-level concurrency utilities.

26. How would you size and protect an executor used for downstream calls?

Strong answer should cover: Use workload type, concurrency budget, latency, arrival rate, queue bounds, rejection policy, timeouts, bulkheads, metrics, and graceful shutdown.

27. Compare synchronized, ReentrantLock, read-write locks, semaphores, and virtual threads.

Strong answer should cover: Choose by semantics; discuss mutual exclusion, interruptibility, fairness, condition variables, limited resources, pinning, and measurement.

28. How do deadlock, livelock, starvation, and race conditions differ?

Strong answer should cover: Define each with symptoms; use lock ordering, timeouts, reduced shared state, thread dumps, stress tests, and concurrency observability.

29. How do you investigate a sudden CPU spike in a Java service?

Strong answer should cover: Correlate traffic and deployments; inspect CPU, throttling, GC and threads; take repeated thread dumps or profiles; locate hot paths; reproduce and verify.

30. What is your approach to Java performance tuning?

Strong answer should cover: Set a user-facing objective; establish baselines; profile production-like load; change one variable; account for warmup/JIT; verify tail latency and regressions.

Spring and Spring Boot

31. How does Spring create and wire beans, and where can lifecycle surprises occur?

Strong answer should cover: Component discovery, configuration classes, dependency resolution, scopes, post-processors, proxies, initialization order, circular dependencies, and shutdown.

32. Why is constructor injection usually preferable to field injection?

Strong answer should cover: Explicit required dependencies, immutability, testability, null safety, visibility of design problems, and easier creation outside the container.

33. How do Spring proxies affect transactions, caching, security, and self-invocation?

Strong answer should cover: Explain proxy boundaries, interface/class proxies, final methods, internal calls bypassing advice, ordering, and moving boundaries or self-proxy alternatives.

34. Explain propagation and isolation for a Spring transaction in a real workflow.

Strong answer should cover: Map propagation to call boundaries; isolation to anomalies; discuss connection use, rollback rules, remote calls, retries, and transaction duration.

35. What can go wrong with @Async or scheduled jobs in a Spring Boot service?

Strong answer should cover: Unbounded executors, lost context, self-invocation, exception handling, duplicate execution, clock/time zone, overlap, shutdown, and cluster coordination.

36. How do you design externalized configuration safely across environments?

Strong answer should cover: Typed configuration, validation at startup, secret stores, precedence, immutable deployment artifacts, environment promotion, rotation, and no secret logging.

37. What belongs in a Spring Boot Actuator and observability setup?

Strong answer should cover: Health readiness/liveness, metrics, structured logs, traces, build info, sanitized config, secured endpoints, custom indicators with dependency semantics.

38. How do you keep a Spring Boot application startup fast and reliable?

Strong answer should cover: Remove accidental classpath weight, inspect auto-configuration, avoid remote startup dependencies, validate config early, use lazy initialization selectively, measure.

39. How would you test a Spring Boot controller, service, repository, and full request path?

Strong answer should cover: Use unit tests, focused slices, repository integration with real database behavior, contract tests, and a small number of full-context tests.

40. How do you upgrade Spring Boot across a large estate safely?

Strong answer should cover: Inventory versions, read migration notes, automate dependency updates, use compatibility tests, canaries, observability, staged rollout, and shared platform support.

Microservices and Distributed Systems

41. When should a system remain a modular monolith instead of becoming microservices?

Strong answer should cover: Use domain boundaries, team topology, independent scaling/deployment need, operational maturity, change coupling, consistency needs, and total complexity.

42. How do you identify and validate service boundaries?

Strong answer should cover: Start with business capabilities and bounded contexts; examine data ownership, change cadence, invariants, coupling, team ownership, and evolutionary seams.

43. How do you handle a business operation that spans several services?

Strong answer should cover: Avoid distributed ACID by default; use sagas, local transactions, durable events, compensations, idempotency, state machines, and observable recovery.

44. Explain timeout, retry, circuit breaker, and bulkhead interactions.

Strong answer should cover: Budget end-to-end latency; retry only safe transient failures with jitter and caps; prevent retry storms; isolate resources; make degraded behavior explicit.

45. What does idempotency mean for an API or message consumer?

Strong answer should cover: Define operation identity, deduplication store or natural key, atomic result recording, expiry, concurrent duplicates, response replay, and side-effect handling.

46. How do you evolve a service contract without coordinating every deployment?

Strong answer should cover: Backward-compatible additive change, tolerant readers, consumer contracts, version policy, dual read/write when needed, telemetry, and staged removal.

47. How do you manage data ownership and reporting across microservices?

Strong answer should cover: One owner for writes; APIs/events for sharing; replicas or analytical stores for reads; lineage, privacy, freshness, reconciliation, and no shared-table coupling.

48. What failure modes appear only at distributed-system scale?

Strong answer should cover: Partial failure, delay, duplication, reordering, split brain, clock skew, stale reads, overload, coordinated retries, regional failure, and ambiguous outcomes.

49. How do you diagnose an intermittent cross-service latency problem?

Strong answer should cover: Start with traces and correlation IDs; split latency by hop; inspect queues, connection pools, DNS/TLS, retries, saturation, dependency percentiles, and deploy changes.

50. How would you safely decompose a legacy monolith?

Strong answer should cover: Baseline behavior, create seams, extract by business capability, use strangler routing, protect data consistency, add contracts, migrate incrementally, retain rollback.

API Design and Security

51. Design a production-grade REST API for a mutable business resource.

Strong answer should cover: Resource model, methods, status codes, validation, optimistic concurrency, idempotency, pagination, errors, authorization, audit, and versioning.

52. How do you choose between REST, gRPC, asynchronous messaging, and GraphQL?

Strong answer should cover: Compare coupling, latency, streaming, schemas, browser support, caching, discoverability, operational tooling, and consumer-driven query needs.

53. How should API pagination behave when data changes during traversal?

Strong answer should cover: Compare offset and cursor/keyset pagination; define deterministic ordering and tie-breakers; handle snapshot needs, filters, limits, and cursor opacity.

54. What makes an API error model useful to clients and operators?

Strong answer should cover: Stable machine code, safe human message, field errors, correlation ID, retryability, HTTP mapping, documentation, and no internal or secret leakage.

55. How do you secure service-to-service communication?

Strong answer should cover: Workload identity, short-lived credentials, TLS, authorization by audience/scope, secret rotation, network controls, replay resistance, audit, and least privilege.

56. How do authentication and authorization differ, and where should each be enforced?

Strong answer should cover: Identity verification versus permission decisions; edge plus service enforcement; resource-level checks; default deny; policy consistency and auditable decisions.

57. What API security issues do you explicitly review?

Strong answer should cover: Broken object authorization, injection, mass assignment, SSRF, unsafe deserialization, rate abuse, sensitive data exposure, weak tokens, and insecure defaults.

58. How do you implement rate limiting for a multi-tenant API?

Strong answer should cover: Define identity and quota; token/leaky bucket; local versus distributed counters; burst policy; fairness; fail-open/closed; response headers; and observability.

59. How do you handle personally identifiable or regulated data in backend systems?

Strong answer should cover: Minimize collection, classify, encrypt, tokenize where useful, restrict and audit access, control retention/deletion, mask logs, and test data handling.

60. How do you deprecate an API used by unknown or slow-moving consumers?

Strong answer should cover: Usage telemetry, owner discovery, notice period, migration guide, compatibility bridge, explicit sunset signals, escalation, and objective removal criteria.

System Design

61. How do you begin a system design interview?

Strong answer should cover: Clarify users, core use cases, scale, SLOs, consistency, security, geography, cost, and exclusions before drawing components.

62. How do you estimate capacity without false precision?

Strong answer should cover: State assumptions; estimate requests, storage, bandwidth, concurrency and growth; use ranges; find dominant constraints; revisit when architecture changes.

63. Design a URL shortening service.

Strong answer should cover: ID generation, redirect path, custom aliases, storage/indexes, cache, abuse controls, expiry, analytics, availability, hot keys, and regional routing.

64. Design a high-volume notification platform for email, SMS, and push.

Strong answer should cover: Preferences, templates, scheduling, queues, channel adapters, provider failover, retries, dedupe, rate limits, status model, observability, and compliance.

65. Design an order and payment workflow that avoids double charging.

Strong answer should cover: State machine, idempotency key, payment provider ambiguity, outbox/events, inventory reservation, compensation, reconciliation, audit, and manual recovery.

66. Design a real-time inventory service for many stores.

Strong answer should cover: Source of truth, reservation model, consistency by operation, partitioning, event updates, cache, oversell policy, reconciliation, and degraded behavior.

67. Design an audit logging platform with tamper evidence and search.

Strong answer should cover: Event schema, identity and correlation, append-only ingestion, integrity chain or immutability controls, access separation, indexing, retention, privacy, and export.

68. Design a multi-tenant SaaS backend.

Strong answer should cover: Tenant identity propagation, isolation models, noisy-neighbor controls, data partitioning, encryption, customization boundaries, metering, operations, and migration.

69. Design a file processing pipeline for large uploads.

Strong answer should cover: Direct object upload, checksums, metadata state, malware scan, event trigger, chunking, idempotent workers, retries/DLQ, backpressure, result delivery, and cleanup.

70. How do you close a system design interview effectively?

Strong answer should cover: Walk critical flows and failures; verify scale/SLOs; name trade-offs and bottlenecks; cover security/operations/cost; propose evolution and validation.

AWS Cloud Architecture

71. How do you choose among EC2, ECS, EKS, and Lambda for a Java workload?

Strong answer should cover: Compare execution model, cold start, control, team skill, portability, scaling, networking, observability, cost shape, and operational burden.

72. Design a highly available Java service across AWS Availability Zones.

Strong answer should cover: Stateless compute, load balancing, health checks, multi-AZ data tier, autoscaling, resilient dependencies, deployment strategy, backups, and failure testing.

73. How would you design for a regional AWS outage?

Strong answer should cover: Set RTO/RPO first; choose backup/restore, pilot light, warm standby, or active-active; cover data replication, DNS/routing, dependencies, testing, and cost.

74. How do security groups, network ACLs, route tables, and private endpoints differ?

Strong answer should cover: Explain stateful instance/ENI controls, stateless subnet controls, routing, service access without public internet, layered design, and troubleshooting.

75. How do you manage IAM for applications and engineers?

Strong answer should cover: Roles and federation, short-lived credentials, least privilege, permission boundaries, separation of duties, resource policies, access analysis, and audit.

76. When would you choose RDS/Aurora, DynamoDB, ElastiCache, OpenSearch, or S3?

Strong answer should cover: Match access patterns, transactions, scale, latency, query flexibility, durability, consistency, operations, retention, and cost.

77. How do SQS, SNS, EventBridge, Kinesis, and MSK differ?

Strong answer should cover: Queue versus pub/sub versus event routing versus stream; ordering, replay, throughput, retention, delivery semantics, ecosystem, and operational ownership.

78. What does a secure AWS account and environment structure look like?

Strong answer should cover: Organization and accounts by boundary, SCPs, centralized identity/logging/security, guarded networking, budgets, break-glass, and automated baseline controls.

79. How do you control AWS cost without compromising reliability?

Strong answer should cover: Tag/allocate spend; right-size from utilization; autoscale; storage lifecycle; commitments for steady use; architecture cost reviews; anomaly alerts; unit economics.

80. How do you make an AWS architecture observable and auditable?

Strong answer should cover: Metrics/logs/traces, CloudTrail and configuration history, centralized protected logs, correlation, actionable alarms, SLOs, dashboards, and retention.

Relational Databases and Persistence

81. How do you choose indexes for a slow query?

Strong answer should cover: Use the execution plan and real predicates; consider selectivity, column order, covering indexes, joins/sorts, write cost, bloat, and representative data.

82. Explain transaction isolation anomalies and how you select an isolation level.

Strong answer should cover: Dirty/non-repeatable/phantom reads, lost updates, write skew, database-specific behavior, locking/MVCC, retries, and business invariants.

83. Optimistic or pessimistic locking: when would you choose each?

Strong answer should cover: Use contention and conflict cost; discuss version columns, retries, lock duration/order, deadlocks, user experience, and hot records.

84. How do you diagnose database connection pool exhaustion?

Strong answer should cover: Inspect active/idle/wait metrics, query time, leaked connections, transaction scope, downstream stalls, pool versus DB limits, timeouts, and load.

85. What causes the N+1 query problem, and how do you fix it safely?

Strong answer should cover: ORM navigation and lazy loading; prove with query counts; use joins/entity graphs/batching/projections; avoid Cartesian explosions; test expected SQL behavior.

86. How do you evolve a large table with zero or minimal downtime?

Strong answer should cover: Backward-compatible expand/contract, online DDL capabilities, batched backfill, dual compatibility, validation, observability, throttling, and rollback.

87. How do normalization and denormalization affect service design?

Strong answer should cover: Normalize for integrity and change safety; denormalize for measured read needs; define ownership, update path, consistency, and reconciliation.

88. How do you implement an outbox pattern correctly?

Strong answer should cover: Write business state and outbox atomically; relay with ordering/partition choice; tolerate duplicate publish; idempotent consumer; cleanup; lag and failure monitoring.

89. How do you approach database partitioning or sharding?

Strong answer should cover: Start with access patterns and growth; choose stable key; balance distribution/locality; handle resharding, cross-shard queries, uniqueness, transactions, and operations.

90. What belongs in a database backup and recovery strategy?

Strong answer should cover: Business RPO/RTO, full/incremental/log backups, encryption, retention, cross-region/account copies, restore automation, integrity checks, and regular drills.

Kafka and Event-Driven Systems

91. Explain topics, partitions, offsets, consumer groups, and replication.

Strong answer should cover: Relate partition to ordering/parallelism, offset to consumer progress, group to work sharing, replicas to durability/availability, and leaders to writes.

92. How do you choose a Kafka message key and partition count?

Strong answer should cover: Use ordering and locality needs; assess key distribution and hot keys; calculate throughput/concurrency headroom; consider future growth and repartitioning cost.

93. What delivery guarantees can Kafka provide end to end?

Strong answer should cover: Separate broker delivery from business effect; at-most/at-least/exactly-once processing; idempotence and transactions; external side effects still need idempotency.

94. How do you build an idempotent Kafka consumer?

Strong answer should cover: Stable event ID or business key; atomic processed marker plus state change; unique constraints; safe retries; retention; partition concurrency; and duplicate metrics.

95. What causes consumer lag, and how do you respond?

Strong answer should cover: Traffic surge, slow processing, downstream latency, rebalances, insufficient partitions, GC/CPU, poison records; measure per partition; scale or remove bottleneck safely.

96. How do you handle poison messages and repeated processing failures?

Strong answer should cover: Classify transient versus permanent; bounded retries with backoff; preserve context; parking/DLQ policy; alerting; replay tooling; ownership and data correction.

97. How do you evolve Kafka event schemas?

Strong answer should cover: Schema registry and compatibility rules; additive defaults; separate semantic from structural compatibility; producer/consumer rollout order; versioning; replay testing.

98. How do rebalances affect consumers, and how do you reduce disruption?

Strong answer should cover: Membership and assignment, stop-the-world risk, processing/heartbeat limits, cooperative strategies, static membership, graceful shutdown, bounded poll work.

99. When is Kafka the wrong choice?

Strong answer should cover: Simple task queues, low-volume point-to-point work, request/response, strict cross-system transactions, limited operational capability, or when a managed event service fits better.

100. How do you operate Kafka securely and reliably?

Strong answer should cover: TLS and authentication, ACLs, least privilege, quotas, replication/ISR, retention, capacity, broker and client metrics, backups where required, upgrades, and failure drills.

CI/CD, Containers, and Platform Engineering

101. What stages belong in a production CI/CD pipeline for Java services?

Strong answer should cover: Fast compile/unit/static/security checks; artifact and SBOM; integration/contract tests; immutable promotion; deploy verification; approvals by risk; rollback evidence.

102. How do you make a pipeline fast without weakening confidence?

Strong answer should cover: Parallelize independent checks, cache safely, test impacted scope, split fast gate from deeper suites, eliminate flakes, reuse immutable artifacts, and measure queue time.

103. How do blue-green, canary, and rolling deployments differ?

Strong answer should cover: Compare blast radius, capacity, traffic control, state compatibility, observability, rollback speed, and when each matches risk and platform capability.

104. What makes a container image production ready?

Strong answer should cover: Minimal trusted base, pinned and scanned dependencies, non-root user, read-only intent, correct signals, resource defaults, no secrets, provenance, and reproducible build.

105. How do readiness, liveness, and startup probes differ?

Strong answer should cover: Readiness gates traffic, liveness restarts stuck processes, startup protects slow initialization; avoid dependency-induced restart storms and overly sensitive probes.

106. How do you set CPU and memory requests and limits for Java containers?

Strong answer should cover: Use measured workload, container-aware heap/native budget, GC behavior, CPU throttling, QoS, autoscaling signals, headroom, and load validation.

107. What should be managed as infrastructure as code?

Strong answer should cover: Cloud/platform resources, policy, observability and delivery primitives; modular but understandable code; reviewed plans; state protection; drift detection; tested changes.

108. How do you manage secrets through build and deployment?

Strong answer should cover: Never bake or log them; use workload identity and secret manager; least access; short lifetime/rotation; masked pipeline variables; audit; safe local development.

109. How do you prevent a bad database migration from blocking deployment?

Strong answer should cover: Separate compatible schema change from code switch; preflight; timeouts/locks; online operations; throttled backfill; progress metrics; rollback or roll-forward plan.

110. What are software supply-chain controls you expect?

Strong answer should cover: Trusted sources, pinned versions, lockfiles, SBOM, vulnerability and license policy, artifact signing/provenance, isolated builds, protected branches, and controlled promotion.

Reliability, Observability, and Testing

111. How do SLI, SLO, SLA, and error budget differ?

Strong answer should cover: Measure user-visible reliability; set objectives and contractual commitments separately; use error budget to guide release/risk decisions and improvement work.

112. What telemetry would you add to a new backend service?

Strong answer should cover: Structured logs, RED/USE metrics, traces, correlation and tenant context, dependency signals, business outcomes, version/config identity, and careful cardinality.

113. How do you design alerts that an on-call engineer can act on?

Strong answer should cover: Alert on user impact or imminent exhaustion; include scope, evidence, runbook and ownership; tune windows; avoid duplicate/noisy symptoms; test routing.

114. Describe your first 15 minutes during a severe incident.

Strong answer should cover: Establish command and communications; assess impact; stop risky change; use known mitigation; preserve evidence; delegate investigation; provide timed updates.

115. How do you run a useful post-incident review?

Strong answer should cover: Blameless timeline, contributing technical/organizational factors, detection and response gaps, well-owned actions, priority and due dates, learning distribution, follow-through.

116. What is your testing strategy for a microservice?

Strong answer should cover: Test pyramid/diamond by risk: pure logic units, component integration with real dependencies, contracts, selective end-to-end, performance/resilience/security tests.

117. How do consumer-driven contract tests help, and what can they miss?

Strong answer should cover: Protect interaction compatibility and independent deployability; govern consumers; still test semantics, infrastructure, end-to-end flows, and non-functional behavior.

118. How do you test retry, timeout, and circuit-breaker behavior deterministically?

Strong answer should cover: Controlled fake server or fault proxy, virtual time where possible, bounded attempts, asserted timings/states/metrics, no sleeps, and isolated test state.

119. How do you prepare for capacity growth?

Strong answer should cover: Forecast from business drivers and unit economics; load-test realistic mixes; identify saturation; set scaling triggers; validate dependencies; plan headroom and degradation.

120. How do you decide whether to roll back or roll forward?

Strong answer should cover: Time to mitigate, data/schema compatibility, reversibility, active user impact, confidence in fix, partial rollout, security exposure, and explicit decision authority.

Coding and Problem Solving

121. Implement an LRU cache and explain the thread-safety trade-offs.

Strong answer should cover: Hash map plus doubly linked list for $O(1)$; define capacity/eviction; handle updates; then compare coarse lock, segmented design, and production library use.

122. Given a stream of events, find the top K keys in a sliding time window.

Strong answer should cover: Clarify exact versus approximate; event versus processing time; heap/count map or sketches; expiration buckets; out-of-order data; memory and concurrency.

123. Merge overlapping intervals and discuss invalid or unbounded inputs.

Strong answer should cover: Sort by start, sweep and merge; $O(n \log n)$; define touching boundaries; validate endpoints; avoid overflow; consider already sorted and streaming variants.

124. Design a bounded blocking queue.

Strong answer should cover: Lock and condition semantics or monitor loops; capacity invariant; spurious wakeups; interruption; fairness; shutdown behavior; tests for blocking and concurrency.

125. Detect and remove duplicates from a large file that does not fit in memory.

Strong answer should cover: Clarify exactness; external sort or hash partitioning; disk and file-descriptor limits; skew; parallelism; checksums; recovery; approximate alternative.

126. Implement a token-bucket rate limiter.

Strong answer should cover: Capacity, refill rate, monotonic time, atomic update, burst semantics, concurrent callers, distributed limitations, tests with fake clock, and overflow safety.

127. Review code that performs parallel downstream calls. What do you inspect?

Strong answer should cover: Deadlines, bounded concurrency, executor ownership, cancellation, context propagation, partial failure, aggregation semantics, resource cleanup, metrics, and tests.

128. How would you process a tree or graph without risking stack overflow?

Strong answer should cover: Iterative DFS/BFS, explicit stack/queue, visited tracking, cycle handling, memory trade-offs, cancellation, and pathological input tests.

129. Write a SQL query to return the latest successful event per customer.

Strong answer should cover: Window function with `row_number` over customer ordered by timestamp plus deterministic tie-breaker; appropriate filter/index; define duplicate timestamp semantics.

130. How do you communicate during a coding interview?

Strong answer should cover: Restate contract, ask about constraints, work from examples, choose approach, state complexity, code in small verifiable steps, test edges, and discuss production refinements.

Behavioral, People, and Delivery Leadership

131. Tell me about the most consequential production incident you led.

Strong answer should cover: Use STAR with scale; show decisions under uncertainty, communication, mitigation, technical diagnosis, accountability, follow-up changes, and measured improvement.

132. Describe a project that missed its goal. What was your role?

Strong answer should cover: Own your decisions; separate causes from excuses; show early signals, stakeholder handling, corrective action, learning, and changed behavior.

133. How have you influenced a decision without formal authority?

Strong answer should cover: Map stakeholders and incentives; build evidence and coalition; listen and adapt; make the decision easy to evaluate; quantify the resulting outcome.

134. How do you mentor senior and junior engineers differently?

Strong answer should cover: Adjust guidance to autonomy and goals; pair and model thinking for juniors; use sponsorship, stretch ownership, feedback, and strategic context for seniors.

135. How do you handle sustained underperformance?

Strong answer should cover: Set explicit observable expectations; seek causes; provide support and timely feedback; document progress; protect the team; involve management fairly and early.

136. How do you delegate critical work without losing control?

Strong answer should cover: Delegate outcomes and decision boundaries; choose capable owner; agree checkpoints and escalation triggers; provide context/resources; avoid taking work back prematurely.

137. Describe a difficult stakeholder conflict.

Strong answer should cover: Show empathy without avoidance; identify shared outcome and constraints; use evidence/options; clarify decision rights; document commitments; preserve the relationship.

138. How do you improve delivery predictability?

Strong answer should cover: Small batches, dependency/risk discovery, clear done criteria, WIP control, forecast ranges, stable teams, quality automation, and honest trend-based communication.

139. How do you build an inclusive engineering culture?

Strong answer should cover: Inclusive meetings and reviews, equitable opportunity and sponsorship, psychological safety, objective expectations, respectful feedback, accessibility, and outcome measurement.

140. Why are you ready for this Technical Lead role?

Strong answer should cover: Connect repeated evidence to role needs: architecture judgment, hands-on depth, delivery, incident leadership, stakeholder influence, talent growth, and business impact.

End-to-End System Design Drills

Run each drill in 45 minutes: 5 minutes to clarify, 5 to estimate and define SLOs, 20 to design critical flows, 10 to test failures and trade-offs, and 5 to summarize. Draw data ownership and failure boundaries, not just boxes.

Drill 1: Global checkout and payment platform

Design checkout for 20k orders/second across regions with inventory reservation, multiple payment providers, fraud checks, and no duplicate charge.

Coverage checklist: Requirements and SLOs; order state machine; consistency boundaries; idempotency; provider ambiguity; outbox/eventing; data model; regional strategy; reconciliation; observability and security.

Drill 2: Multi-channel notification service

Design scheduled and immediate notifications with tenant-specific limits, user preferences, templates, delivery status, and provider failover.

Coverage checklist: API and schemas; queue/partitioning; dedupe; retry/DLQ; rate control; template/version management; preferences/compliance; adapters; status and callbacks; capacity and cost.

Drill 3: Streaming fraud detection

Score financial events within 200 ms, combine recent behavior with durable history, update rules safely, and support investigation/replay.

Coverage checklist: Event time and ordering; partitions; feature/state stores; rules/models; latency budget; exactly-once claims; late events; replay/versioning; audit; fallback; privacy; operational metrics.

Drill 4: Large-scale document ingestion

Accept multi-GB files, scan and transform them, extract metadata, expose status, and retain or delete data by policy.

Coverage checklist: Direct object upload; multipart/resume; checksum; metadata state machine; event trigger; idempotent tasks; malware isolation; retries; quotas; lifecycle; tenant security; cost.

Drill 5: Feature flag platform

Provide low-latency evaluations for thousands of services, percentage rollouts, targeting, audit, and safe emergency disable.

Coverage checklist: Control/data plane; rule model; deterministic bucketing; cache/stream distribution; stale behavior; SDK contract; consistency; change audit; access control; kill switch; scale.

Drill 6: API rate-limiting service

Enforce per-user and per-tenant quotas across regions with bursts, plan changes, and useful client feedback.

Coverage checklist: Algorithms; identity; local/global trade-off; distributed counters; clock; failure mode; hot keys; configuration propagation; headers; monitoring; fairness; cost.

Drill 7: Operational audit platform

Collect immutable audit events from hundreds of services, retain for years, search recent data quickly, and prove integrity.

Coverage checklist: Schema and identity; durable ingestion; partitioning; tamper evidence; hot/cold storage; indexing; retention/legal hold; access separation; privacy; export; recovery.

Drill 8: Legacy monolith decomposition plan

Plan a 12-month migration of a revenue-critical Spring monolith with one database and weekly releases.

Coverage checklist: Baseline and goals; domain seams; team ownership; strangler routing; database transition; contracts; observability; migration sequence; rollback; risk controls; milestones and success measures.

Self-Scoring Rubrics

Technical question - score each answer from 0 to 4

- 0: No answer or materially incorrect.
- 1: Names concepts but cannot connect them to behavior or consequences.
- 2: Explains the standard solution and one or two trade-offs.
- 3: Clarifies context, compares options, covers failures, security, operations, and testing.
- 4: Adds credible production evidence, metrics, nuanced limits, and a clear recommendation.

System design - six dimensions

- Requirements: clarifies use cases, exclusions, scale, SLOs, consistency, privacy, and cost.
- Architecture: presents coherent components and walks the critical read/write paths.
- Data: defines ownership, schemas, indexes, partitioning, consistency, retention, and migration.
- Reliability: handles partial failure, overload, retries, dedupe, recovery, and disaster scenarios.
- Security and operations: covers identity, authorization, secrets, audit, telemetry, rollout, and support.
- Communication: drives the discussion, states assumptions, manages time, and closes with trade-offs.

Leadership story - evidence checklist

- The stakes and your personal responsibility are unmistakable.
- You describe specific decisions and influence, not only what the team did.
- You show constructive conflict, communication, and stakeholder management.
- The result includes a number, observable change, or defensible business outcome.
- You own mistakes and explain how your later behavior or system changed.

Four-Week Preparation Plan

Week 1 - Baseline and story bank

- Record answers to 20 representative questions. Score them and identify the weakest three domains.
- Prepare eight STAR+ stories: architecture, incident, failure, conflict, influence, mentoring, delivery, and security or quality.
- Refresh Java/JVM, Spring transactions/proxies, SQL plans/indexes, and one coding pattern each day.

Week 2 - Distributed systems and cloud

- Practice microservice failure modes, idempotency, sagas, outbox, contract evolution, and Kafka operations.
- Design two systems under time limits. Include estimates, data model, security, SLOs, deployment, and disaster recovery.
- Map each AWS service you mention to a requirement; avoid technology lists without trade-offs.

Week 3 - Production depth and leadership

- Practice incident, observability, capacity, CI/CD, supply chain, database migration, and resilience questions.
- Run mock leadership rounds. Remove vague claims and replace them with decisions, evidence, and outcomes.
- Prepare concise explanations of your current architecture, biggest scaling limit, and most important technical decision.

Week 4 - Full simulations and refinement

- Complete two full interview loops: coding, Java/Spring depth, system design, and behavioral/leadership.
- Review recordings for structure, pace, filler, unsupported claims, and missed failure/security considerations.
- Prepare role-specific questions, a 90-second introduction, a five-minute flagship project story, and final revision notes.

Suggested daily 90-minute block

20 min recall -> 25 min deep technical answer -> 30 min design or coding -> 10 min leadership story -> 5 min score and schedule the next review.

Final Interview Readiness Checklist

- [] I can explain my current system in five minutes, including scale, data, reliability, security, cost, and my decisions.
- [] I can discuss Java memory, concurrency, executors, GC evidence, and performance diagnosis without hand-waving.
- [] I can explain Spring proxies, transaction boundaries, configuration, testing, and upgrade strategy using real examples.
- [] I can reason about retries, timeouts, idempotency, consistency, contract evolution, and partial failure.
- [] I can design and operate workloads on AWS while covering IAM, networking, multi-AZ, regional recovery, and cost.
- [] I can read a SQL execution plan and discuss indexes, isolation, locking, migrations, pooling, and backup restore.
- [] I can explain Kafka partitioning, ordering, consumer lag, schema evolution, replay, poison messages, and end-to-end semantics.
- [] I can describe an immutable CI/CD path from source to production, including supply-chain and rollback controls.
- [] I can define meaningful SLOs, alerts, dashboards, traces, capacity signals, and incident response expectations.
- [] I have eight concise leadership stories with specific actions, measurable results, and honest learning.
- [] I clarify requirements before proposing technology and explicitly state trade-offs and assumptions.
- [] I end every design with risks, security, operations, testing, rollout, rollback, and how the design evolves.
- [] I can code a correct solution, test edge cases, state complexity, and explain production-grade refinements.
- [] I have thoughtful questions about product goals, architecture ownership, team health, delivery constraints, and success in the role.

Readiness target: consistently score 3 or above on technical answers, cover all six system-design dimensions without prompting, and deliver leadership stories in under three minutes with a measurable result.

Questions to ask the interviewer

- What are the hardest technical and organizational problems this lead should solve in the first six months?
- How are architecture decisions made, documented, and revisited across teams?
- Which reliability, delivery, quality, and business measures define success for this team?
- Where does the team currently feel the most operational or technical friction?
- How much of this role is hands-on engineering, technical direction, people development, and stakeholder alignment?
- What distinguishes Technical Leads who thrive here from those who struggle?